

Zipf's Law: Completeness and Accuracy

Eric Foster

Advisor: Dr. Valerie Sessions,

April 20, 2011

Table of Contents

Senior Project: Zipf's Law: Data Completeness and Accuracy	
Introduction.....	3
Project Background and Intentions	3
Project Summary.....	4
Research and Understanding.....	5
Normal Distribution.....	5
Grubb's Test	5
Power Law Distribution	6
Zipf's Law	6
Explanation of Data Sets	7
Data Interpretation	8
Code Walkthrough	9
Problems and Pitfalls.....	13
Graph of Completeness.....	15
Graph of Dirty	16
The Data	18
All Code	24
Final Presentation	47

Introduction

Project Background and Intention

This project's intention is to find better ways to identify error in a given data set. Typically, you plot data and check to see if it is a normal distribution. Through a normal distribution we can easily see outliers in data and understand that they most likely represent error as they don't fit into the normal distribution. Error in a standard distribution is often easily identifiable. Normal distributions also allow for the use of Grubb's test to determine if a piece of data is an outlier. A more in-depth understanding of probability and statistics enlightens us to realize that data doesn't always fit into a normal distribution. Typically we even assume data to be a normal distribution when it really isn't. Most real-world data sets actually fit a power law distribution. In this project we use the premise that not all data is normal to test data that fits a power law distribution instead. A specific type of power law distribution is a Zipfian distribution. In Zipfian distribution data is reordered. The data is listed in descending order and the log of the ranks is graphed against the log of the specific values. If the resulting graph is linear, then it fits Zipf's law. A power law distribution however doesn't provide a law or

rule to adequately indentify an outlier in data. Through identification of outliers in power law distributions, we can test data quality more thoroughly.

Project Summary

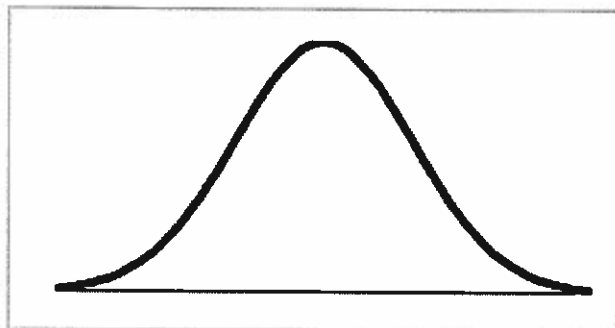
This project uses scripts and the work already done by Chris Nuhn and Dr. Valerie Sessions. They tested data by slightly skewing the data by a percentage and seeing how it affects various aspects of the data, for a chance to analyze error.

They initially only used five data sets and tested by corrupting the data. This project expands upon that work by also testing for completeness of the data and by adding more data to the research. This project also contains my own scripts that test for completeness by removing given percentages of the data to test whether or not there is a measurable outcome to help for the testing of outliers.

This project adds seven more data sets to the research, runs Chris Nuhn's scripts on the new data sets, and furthermore runs my own scripts against all twelve. In the end however my completeness data doesn't appear to provide any insight into understanding outliers for a power law distribution or a Zipfian distribution, but it does provide data for further interpretation and research. Dr. Sessions will continue to study my results and hopefully find a line we can use to determine if a given piece of data is an outlier.

Research and Understanding

Normal Distribution



Above is an example of a normal distribution. We can easily identify outliers within a normal distribution using Grubb's test.

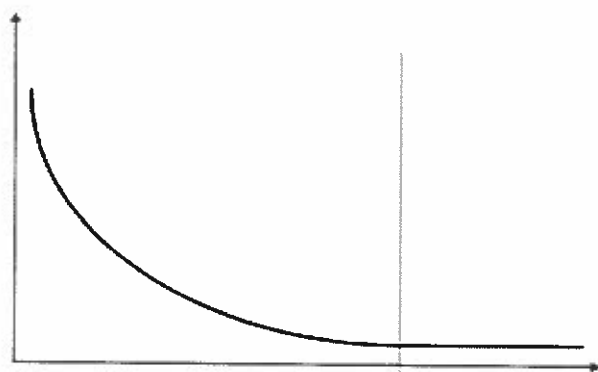
Grubb's Test

$$g = \frac{\bar{x} - x_{\min}}{s} \qquad g = \frac{x_{\max} - \bar{x}}{s}$$

When using Grubb's test you use either of the above two formulas to determine an outlier. To use the test you require the minimum or maximum x value. $\bar{x}$ represents the mean and s represents the standard deviation of the sample.

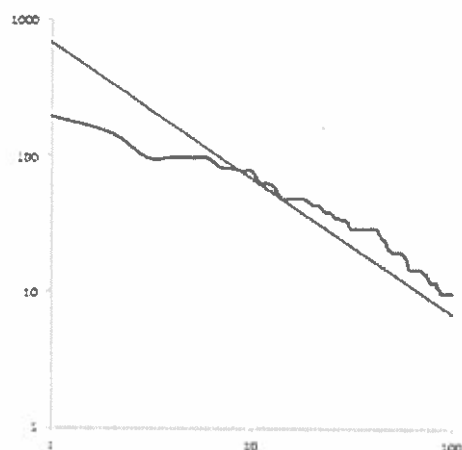
Grubb's Test can only determine one outlier at a time. If g is greater than the critical value of the point you are checking, then you can consider it an outlier. An easier way to consider this is that, if your point isn't between the g of both formulas, then it is an outlier. Basically if the point is outside of the interval $((\bar{x} - x_{\min})/s, (x_{\max} - \bar{x})/s)$ it is an outlier.

Power Law Distribution



The graph above represents a Power Law Distribution. While a normal distribution lent itself to use of Grubb's test, the power law distribution doesn't lend itself to any known test.

Zipf's Law



This graph shows a Zipfian distribution in blue, with the best line approximation in red. As we notice, the blue line is very close to being linear, so we go ahead and naively assume it to be perfectly linear by making an approximation.

Explanation of Data Sets

For testing completeness and building upon the test using data corruption, I used twelve data sets. My sets varied in several factors including: data set size, slope and correlation.

1. Blackouts - the number of customers affected by electrical blackouts in the United States between 1984 and 2002.
2. Cities - The population of US cities in the 2000 national Census.
3. Crime - Crime activity in a Certain US city.
4. Fires - The size in acres of wildfires that occurred in the US on federal land between 1986 and 1996.
5. Flares - The peak gamma-ray intensity of solar flares between 1980 and 1989.
6. Citations - the number of citations received between publication and June 1997 by scientific papers published in 1981 and listed in the Science Citation Index.
7. MOM - the number of species per genus of recent mammals.
8. Names - The frequencies of US family names in the 1990 United States Census.

9. Quakes - intensity of earthquakes in California from 1910 to 1992, measured as the maximum amplitude during the quake.
10. Terror - the severity of terror attacks globally from February 1968 to June 2006.
11. Web - The number of links to websites found in a 1997 web crawl of around 200 million web pages.
12. Words - the frequency of unique words in the novel *Moby Dick*.

Data Interpretation

The correlation and slope were the two prime factors we analyzed for a pattern. After adding my new data sets we found that the dirty data continued to follow the trend found in the first experiment with Chris's project. After corrupting the completeness of the data however, no clear pattern showed, and as the data became less complete, the data spread was significant. While there is no promise of a pattern to help us derive a line in which we can easily identify outliers yet, we have provided data that can continue to be analyzed. Dr. Sessions will continue to examine this data and after further analysis may locate a trend that reveals how to identify outliers for a Zipfian distribution.

Code Walkthrough

```
foreach(.01,.02,.03,.04,.05,.06,.07,.08,.09,.1,.11,.12,.13,.14,.15,.20,.25,.30,.35,.40,.45,.50)
{
    $amount = int($rank * $_);
    for($l=0; $l<$rank; $l++)
    {
        $new[$l]=$y[$l];
    }
    for($i = 0; $i<$amount; $i++)
    {
        my $np;
        do
        {
            $np = int(rand($rank));
        } until ($new[$np] ne " ");
        $new[$np]=" ";
    }
    chdir("$first");
    chdir('complete');
    open( OUT, ">$_.\"$first\".complete\".txt");
    foreach (@new)
    {
        print OUT $_, "\n";
    }
    close(OUT);
}
```

The above snippet of code is from my script complete.pl. My foreach loop uses percentages from 1 to 15, then from 20 to 50 by fives. Using these percentages I take out data to change its completion. \$Rank represents a counter I basically use as an index. Prior to this foreach loop, \$rank was incremented and now represents the number of data in the set. By setting \$amount to \$rank times the given decimal, I can track how many pieces of data I now need to change. \$New represents an array I use to duplicate the original array \$y of data. I need to maintain \$y for each loop through so I still have the original data set for manipulation. The for loop then randomly removes \$_ percentage of data. The do until loop simply checks to make sure the random spot hasn't already been picked, and if it has will continue

to generate a random spot until it chooses a spot not previously picked. This ensures that I truly remove the given percentage. For example: If I want to remove 20% of a data set of size ten, I want to make sure I remove two values, and not remove the same one twice, only really removing 10%. After removing data, I then change directories to store the corrupt data in a new file.

```
chdir 'C:/Users/Eric/Desktop/NewProjectAttempt/data';

$file = $ARGV[0];

open(IN,$file) or die;
@data=<IN>;
close(IN);

($first, $extension) = split /\./, $file;
open(OUT,">$first". "convert.txt") or die;#c
foreach $dt (@data)
{
    my @converts = split(/\s/, $dt);
    foreach $cts (@converts)
    {
        print OUT ($cts * 100);
        print OUT " ";
    }
    print OUT "\n";
}
unlink($file);
print "done...";
```

The next snippet of code is from convert.pl. I simply used this for converting decimal data to help Chris Nuhn's scripts handle the data. Prior to this conversion, Chris's scripts would crash. This code takes a file, \$file, as a parameter. It then opens that file and pushes the data into the array \$data. I split the file into its name and extension. I then create an output

file. You may notice the split on /s I use to create the array of converts. I do this to only consider the data on the line, and ignore any extra tabbing, spacing, etc that was at the end of the file. I found that several data sets had issues with random spaces at the end of the data; probably just how someone put them in by accident, but it did cause issues. I then multiply the data by 100, or a large number canceling out the decimals. The multiplication I do now will fall out later when I interpret the data so it doesn't matter. Finally `$unlink($file)` will delete the original file only leaving the new converted file.

```
print "starting the execution...\n";
opendir DIR, 'C:\Users\Eric\Desktop\NewProjectAttempt\data\cities\complete' or die "open failed : $!\n"; $just setting the directory for where data will be.
for(readdir DIR)
{
    if($ _ =~ m/\.txt$/) # if the file is indeed a txt file.
    {
        'perl C:/Users/Eric/Desktop/NewProjectAttempt/research.pl $ _ 0 0 cities 1';
        {
            'perl C:/Users/Eric/Desktop/NewProjectAttempt/complete.pl $ _'; #execute complete.pl over all data sets.
            print "execution on $ _ finished!";
        }
        'perl C:/Users/Eric/Desktop/NewProjectAttempt/dirty.pl $ _'; #execute dirty.pl over all files in directory.
        'perl C:/Users/Eric/Desktop/NewProjectAttempt/dirty-pt2.pl $ _'; #execute dirty-pt2.pl over all files in directory.
        ($first, $extension) = split /\./, $ _; #split the file $split off the file name.
        $dir2 = 'C:\Users\Eric\Desktop\NewProjectAttempt\data\' . $first . "\complete\"; #save the directory that will be named after the first part of the filename.
        opendir(DIR2, $dir2) or die "open failed : $!\n";
        for(readdir DIR2)
        {
            if($ _ =~ m/\.txt$/) #make sure you execute only on text files and not folders.
            {
                'perl C:/Users/Eric/Desktop/NewProjectAttempt/research.pl $ _ 0 0 $first 1'; #run research.pl across all data in directory.
            }
        }
        closedir DIR2 or die "close failed: $!\n";

        $dir3 = 'C:\Users\Eric\Desktop\NewProjectAttempt\data\' . $first . "\dirtypt2\"; #store directory where dirtypt2 data will be stored per data set.
        opendir(DIR3, $dir3) or die "open failed : $!\n";
        for(readdir DIR3)
        {
            if($ _ =~ m/\.txt$/) #make sure it is a text file.
            {
                'perl C:/Users/Eric/Desktop/NewProjectAttempt/research.pl $ _ 0 0 $first 1'; #execute across all dirtypt2 data in directory.
            }
        }
        closedir DIR3 or die "close failed: $!\n";
    }
}
closedir DIR or die "close failed : $!\n";
print "execution finished!\n";
exit(0);
```

The snippet of code on the previous page is for my `execute.pl` script. I used this script to automate my creation of data using `dirty.pl`, `dirty-pt2.pl`, and `complete.pl`. It also automates the gathering of correlation coefficients and slopes using `research.pl`. I use this script to iterate through all files in a given directory, and it switches directories and also saves the created data in their appropriate directories. I also added two extra parameters to Chris Nuhn's `research.pl` to make directory changing and iteration over files within a directory easier. I use a pattern match on the files inside the directory to determine if they are text data files I am looking for instead of a script or folder.

Problems and Pitfalls

Using existing code

Perl isn't the easiest code to read and makes it even harder to read someone else's code. Chris Nuhn's Perl scripts were not really commented at all and didn't have a very thorough breakdown, so there was a slight learning curve to start. After finally gaining a grasp of Chris's code I was able to proceed.

Format

The original Perl scripts didn't handle decimal data at all. To deal with this problem I simply multiplied the data set by a large number such as 100,000 to remove the decimal. Later on when I used the data, the multiplied number would fall out anyways. This conversion was done in my script `convert.pl` which took the file to be converted, converted the data, wrote it to a new file, and deleted the old.

Running Across Data Sets

The original Perl scripts I was provided had to be manually run across data which proved to be tedious. I simply created `execute.pl` and used it to execute the already existing scripts, plus my scripts, across data, while switching directories and saving data in the appropriate folders.

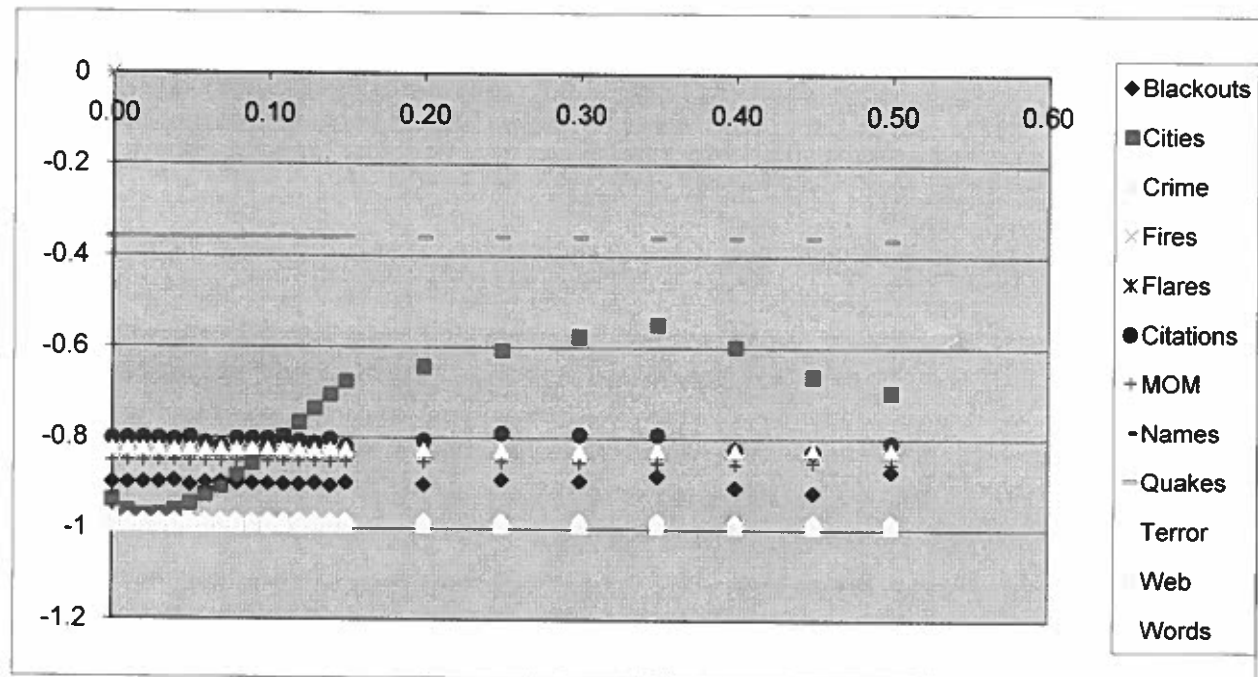
Large Data sets and Memory Issues

While attempting to run my complete.pl script over the files, it encountered a lockup on the firesconvert.txt data set. Upon investigation I realized it contained over 200,000 data entries and so the arrays in Perl couldn't index that high and would crash. I simply skipped this file. Further research may find it more efficient to break up the file to compensate for data overflow.

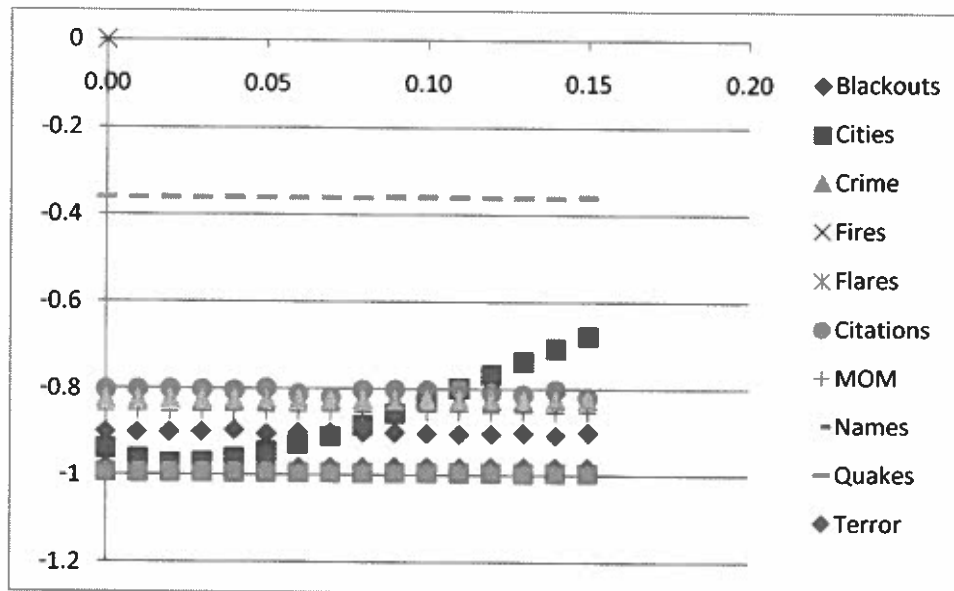
Locating Data

One of the most challenging parts of this research involved actually locating power law data sets that fit a Zipfian distribution. Some data sets I found actually were in formats that were far too complicated to read and nearly impossible to extract. Some data sets I found also relied on other research and data that wasn't available.

Graph of Completeness Results

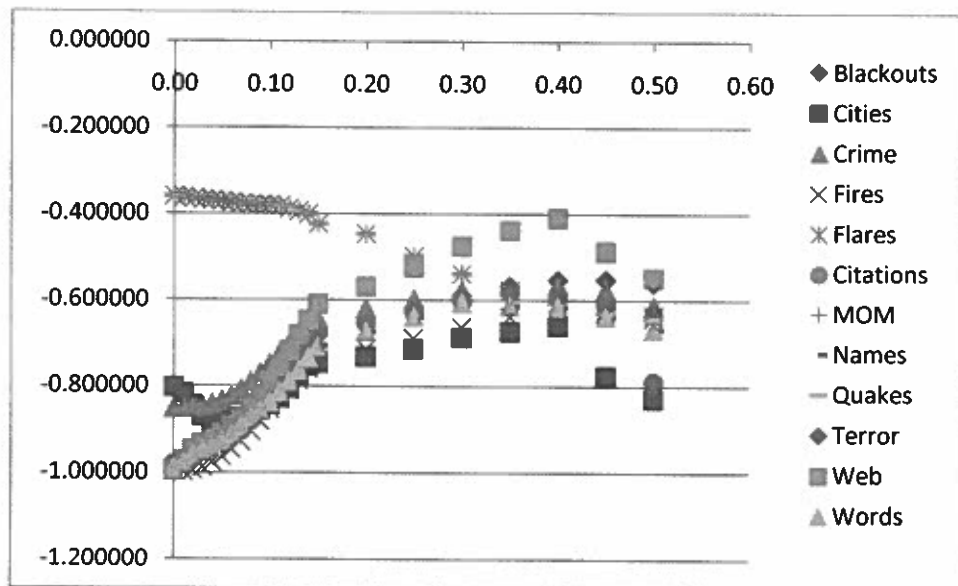


Fires data doesn't show up on this graph as it was removed since the data didn't actually get computed. Flare data is in the graph but so closely follows web and terror data that it can hardly be seen. The quakes data is completely off from the rest, but this is because this it isn't a confident power law or Zipfian distribution.

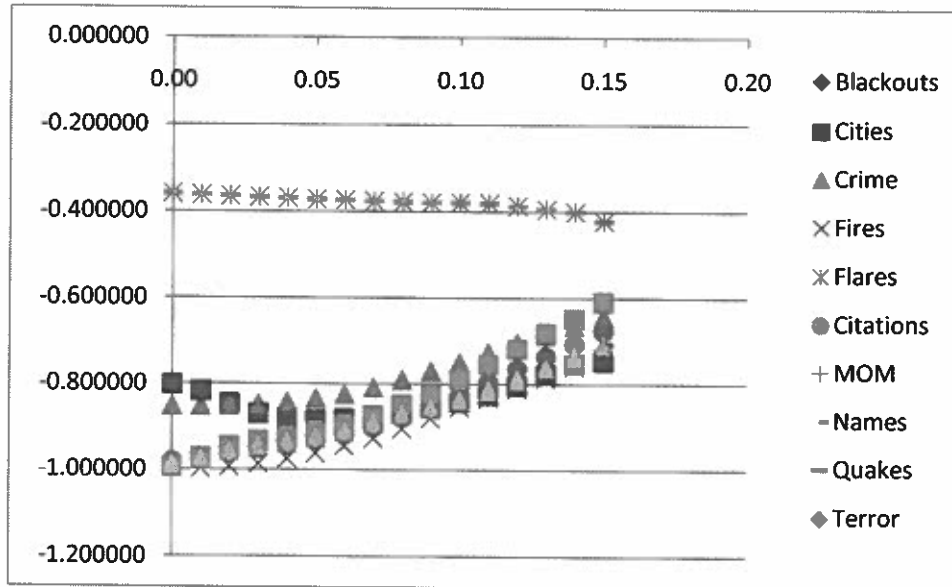


This second graph is my completeness data, which shows a tighter spread when reduced to the 0-15% data removal. Again quakes data doesn't match the rest of the data, and fires data still doesn't appear as it was far too large.

Graph of dirty data



The data for dirty data lines up nicely except for quakes data which yet again appears as it was the weakest power law amongst the sets. As the interval is reduced yet again from 0-15%, we notice a much tighter correlation.



This above graph shows the tightness as the interval is reduced and provides more insight.

The Data

	Blackouts	Cities	CrimeActivity	FiresConvert
Percent Corrupt	r squared value	r squared value	r squared value	r squared value
0.00	-0.900355674	-0.939660416	-0.987120	-0.984268578
0.01	-0.905080045	-0.962893938	-0.985670	-0.988172004
0.02	-0.924598227	-0.972839244	-0.980475	-0.978419157
0.03	-0.9301089	-0.970957793	-0.969894	-0.966465317
0.04	-0.931024352	-0.961354486	-0.965936	-0.955638997
0.05	-0.924987161	-0.94606901	-0.954617	-0.94634907
0.06	-0.916733489	-0.927163483	-0.956083	-0.943367689
0.07	-0.902386619	-0.908123286	-0.960168	-0.939677217
0.08	-0.891673743	-0.884151172	-0.957968	-0.935213952
0.09	-0.867405374	-0.856482804	-0.954160	-0.935889004
0.10	-0.865898018	-0.827828412	-0.948233	-0.933596738
0.11	-0.8483656	-0.798122643	-0.942405	-0.9305809
0.12	-0.83927948	-0.766471534	-0.922567	-0.93788374
0.13	-0.826773903	-0.735702156	-0.895763	-0.948159068
0.14	-0.815228897	-0.705240323	-0.855710	-0.944521991
0.15	-0.801206526	-0.67629171	-0.841892	-0.937397024
0.20	-0.7770092	-0.642545804	-0.806512	-0.933560996
0.25	-0.769591227	-0.607061525	-0.781685	-0.931174638
0.30	-0.74556053	-0.574506046	-0.754964	-0.941011019
0.35	-0.741509705	-0.548113067	-0.722635	-0.956780572
0.40	-0.71942627	-0.597211356	-0.706415	-0.972968053
0.45	-0.689693908	-0.661611467	-0.837400	-0.980795855
0.50	-0.789400769	-0.694275581	-0.929204	-0.986233793
p values	0.62	0.76	NA	0.05
size (n values)	211	19447	201	203785

Dirty

	Flares	ISlcitation	MOM_v3_3	names
Percent Corrupt	r squared value	r squared value	r squared value	r squared value
0.00	-0.992570	-0.802520003	-0.851566619	-0.994842
0.01	-0.977897	-0.816842677	-0.851867509	-0.997266
0.02	-0.960423	-0.845369831	-0.850241198	-0.993554
0.03	-0.948668	-0.869870777	-0.846818381	-0.987005
0.04	-0.939553	-0.87896415	-0.842169851	-0.975075
0.05	-0.928075	-0.884082293	-0.834086265	-0.962358
0.06	-0.913613	-0.881170104	-0.823116679	-0.945028
0.07	-0.895342	-0.875744885	-0.807395204	-0.926630
0.08	-0.872613	-0.86946679	-0.78962173	-0.903462
0.09	-0.847017	-0.854422201	-0.770003638	-0.879958
0.10	-0.819033	-0.842538757	-0.749585139	-0.853686
0.11	-0.790224	-0.826203278	-0.725936088	-0.832537
0.12	-0.760147	-0.803717809	-0.701923151	-0.810317
0.13	-0.729522	-0.777058017	-0.683557149	-0.785785
0.14	-0.700011	-0.751607911	-0.661776853	-0.759144
0.15	-0.670777	-0.746486584	-0.642296365	-0.738759
0.20	-0.641851	-0.732148593	-0.619297506	-0.714443
0.25	-0.615828	-0.713186176	-0.597596384	-0.689070
0.30	-0.587774	-0.686523485	-0.577505119	-0.663526
0.35	-0.567309	-0.672951106	-0.566443221	-0.638208
0.40	-0.552448	-0.659617823	-0.562753809	-0.624336
0.45	-0.551619	-0.776258535	-0.574533691	-0.621493
0.50	-0.559030	-0.828662024	-0.612359987	-0.634528
p values	1	0.2	0.1	0.2
size (n values)	12773	8904	509	2753

	quakesconvert	terrorism	web	wordcount
Percent Corrupt	r squared value	r squared value	r squared value	r squared value
0.00	-0.359633612	-0.981525882	-0.994668283	-0.988579
0.01	-0.361461801	-0.978999647	-0.972187499	-0.977787
0.02	-0.363760013	-0.962976903	-0.946168424	-0.958271
0.03	-0.366786197	-0.951568787	-0.932541161	-0.946614
0.04	-0.369099669	-0.943198633	-0.922267088	-0.937757
0.05	-0.37143976	-0.932191378	-0.910311514	-0.926703
0.06	-0.373502109	-0.918104679	-0.893946815	-0.911636
0.07	-0.376031824	-0.900969897	-0.873170801	-0.892204
0.08	-0.377441099	-0.878312647	-0.848299606	-0.870329
0.09	-0.379057521	-0.852913407	-0.820649252	-0.857065
0.10	-0.379709995	-0.826188721	-0.789120293	-0.837566
0.11	-0.380992395	-0.795845447	-0.753668094	-0.814163
0.12	-0.387575583	-0.764675732	-0.717750563	-0.788848
0.13	-0.393824105	-0.735451183	-0.681543797	-0.762256
0.14	-0.400098998	-0.704903695	-0.646747295	-0.733374
0.15	-0.422247431	-0.675237139	-0.609381205	-0.706243
0.20	-0.446895952	-0.652446682	-0.568402786	-0.673616
0.25	-0.496870116	-0.626749633	-0.523164213	-0.638245
0.30	-0.538721668	-0.599911711	-0.474843447	-0.606023
0.35	-0.576283985	-0.579536212	-0.437770043	-0.612337
0.40	-0.604626785	-0.593710928	-0.408961586	-0.614250
0.45	-0.628417773	-0.60990477	-0.486149778	-0.636806
0.50	-0.642991644	-0.789496447	-0.547552624	-0.667539
P values	0	.68	0	.49
Size (n values)	19302	9201	14480	18855

Completeness

	Blackouts	Cities	CrimeActivity
Percent Removed	r squared value	r squared value	r squared value
0.00	-0.900355674	-0.939660416	-0.987119744
0.01	-0.899958337	-0.962893938	-0.987407744
0.02	-0.899253776	-0.972839244	-0.987117785
0.03	-0.898581437	-0.970957793	-0.987505271
0.04	-0.89599216	-0.961354486	-0.986535727
0.05	-0.905033269	-0.94606901	-0.987740502
0.06	-0.89915726	-0.927163483	-0.986316072
0.07	-0.899318662	-0.908123286	-0.988517592
0.08	-0.901590138	-0.884151172	-0.987344871
0.09	-0.901395714	-0.856482804	-0.988754048
0.10	-0.902180188	-0.827828412	-0.989813879
0.11	-0.903151691	-0.798122643	-0.986464415
0.12	-0.903143405	-0.766471534	-0.987307069
0.13	-0.900750881	-0.735702156	-0.984741761
0.14	-0.906136319	-0.705240323	-0.990688923
0.15	-0.899101318	-0.67629171	-0.983527798
0.20	-0.903068674	-0.642545804	-0.983924431
0.25	-0.89184549	-0.607061525	-0.990412933
0.30	-0.894748534	-0.574506046	-0.988816448
0.35	-0.882665179	-0.548113067	-0.979862572
0.40	-0.90797417	-0.597211356	-0.985673647
0.45	-0.918658084	-0.661611467	-0.989781325
0.50	-0.871500008	-0.694275581	-0.98327049
Extra Information			
p values	0.62	0.76	NA
size (n values)	211	19447	201

	Flares	IScitation	MOM v3 3	names
Percent Removed	r squared value	r squared value	r squared value	r squared value
0.00	-0.992569554	-0.802520003	-0.851566619	-0.994842843
0.01	-0.992627519	-0.801888809	-0.851181918	-0.994789188
0.02	-0.992490698	-0.801080407	-0.851818861	-0.995150615
0.03	-0.992310241	-0.80207799	-0.851327334	-0.994813239
0.04	-0.992370835	-0.804552404	-0.850911685	-0.99484625
0.05	-0.992618414	-0.799378305	-0.85163358	-0.995014635
0.06	-0.992435151	-0.81169291	-0.85181945	-0.994868143
0.07	-0.992517333	-0.819138377	-0.851494199	-0.994670685
0.08	-0.992530328	-0.802154989	-0.851804178	-0.995150017
0.09	-0.992718662	-0.80168543	-0.851660755	-0.995574973
0.10	-0.992197434	-0.802823868	-0.851499476	-0.993349113
0.11	-0.992096436	-0.798789578	-0.851226566	-0.995145846
0.12	-0.992389287	-0.807320487	-0.852058428	-0.994275166
0.13	-0.992398487	-0.812448906	-0.851542318	-0.995447364
0.14	-0.992838203	-0.802359897	-0.852969053	-0.994826284
0.15	-0.992882828	-0.818765012	-0.851567756	-0.994861357
0.20	-0.992819343	-0.805718008	-0.852786769	-0.996157563
0.25	-0.991705864	-0.788698054	-0.852354061	-0.99427406
0.30	-0.993301489	-0.791375154	-0.854111936	-0.996676369
0.35	-0.992446098	-0.791077442	-0.853613521	-0.993800572
0.40	-0.990679883	-0.823092736	-0.856359936	-0.995545882
0.45	-0.993437831	-0.827346354	-0.85197874	-0.997212699
0.50	-0.993172759	-0.80845941	-0.855823483	-0.99575112
Extra Information				
p values	1	0.2	0.1	0.2
size (n values)	12773	8904	509	2753

	quakesconvert	terrorism	weblinks	wordcount
Percent Removed	r squared value	r squared value	r squared value	r squared value
0.00	-0.359633612	-0.981525882	-0.994668283	-0.82390122
0.01	-0.359597114	-0.981360378	-0.994662455	-0.823729468
0.02	-0.359367194	-0.981476158	-0.994687423	-0.82355261
0.03	-0.360366641	-0.98138076	-0.994646826	-0.82388482
0.04	-0.359155871	-0.981642029	-0.994673143	-0.824282065
0.05	-0.360136988	-0.981317206	-0.994585513	-0.824924703
0.06	-0.360012198	-0.981549165	-0.994652836	-0.824393318
0.07	-0.359578204	-0.981238484	-0.99474328	-0.823898596
0.08	-0.360410072	-0.981739951	-0.994592529	-0.824386078
0.09	-0.358298422	-0.981120232	-0.994488452	-0.824148241
0.10	-0.358710627	-0.98173401	-0.994725861	-0.824454443
0.11	-0.35897793	-0.981828128	-0.994536254	-0.823551145
0.12	-0.360211019	-0.980761897	-0.994586956	-0.824747262
0.13	-0.359218587	-0.980648797	-0.994893619	-0.825644036
0.14	-0.360608397	-0.980692027	-0.994593717	-0.824173841
0.15	-0.358638459	-0.9820527	-0.994585904	-0.822869132
0.20	-0.360825822	-0.982029147	-0.994419892	-0.824721853
0.25	-0.359201661	-0.981754884	-0.994812224	-0.825553136
0.30	-0.358723125	-0.980764941	-0.994468816	-0.826661476
0.35	-0.359833533	-0.980842478	-0.994387273	-0.822675836
0.40	-0.359734005	-0.981246105	-0.994660461	-0.823902154
0.45	-0.359258086	-0.981528688	-0.99460749	-0.822780116
0.50	-0.363033363	-0.981309865	-0.994204841	-0.822067694
Extra Information				
p values	0	0.68	0	0.49
size (n values)	19302	9201	14480	18855

```

#Eric Foster and Doctor Sessions

#12:00 AM 03/1/2011

#convert decimal data into integers for ease of use.

#convert.pl

#####

#                ARGS                #

#####

# 1. String      File                #

#####

chdir'C:/Users/Eric/Desktop/NewProjectAttempt/data';

$file = $ARGV[0];#save the file name argument.

open(IN,$file) or die;#open input file and read the data into an array.

@data=<IN>;

close(IN);

($first, $extension) = split /\./, $file;

open(OUT,">$first"."convert.txt") or die;#create an output file to put the converted data into.

foreach $dt (@data)

{

    my @converts = split(/\s/, $dt);#split on spaces and put into array.

    foreach $cts (@converts)#using the array of numbers from the line.

    {

        print OUT ($cts * 100);#multiply them by 100,000 to ignore decimals.

        print OUT " ";#add a space behind each number.

    }

    print OUT "\n";#move to the next line in the output file.

}

```



```
unlink($file);#delete old file that was in the incorrect format.
```

```
print "done...";#obviously done.
```

```
#Eric Foster & Doctor Sessions
```

```
#research algorithm using Zipf's Powerlaw v5.1
```

```
#last edited 6:45pm 04/16/2011
```

```
#complete.pl
```

```
if ($#ARGV >= 0) {$file = $ARGV[0]; $error = "$file not found\n";}
```

```
if ($#ARGV < 0) {$error = "file not given\n";}
```

```
chdir('data');#added for ease of use.
```

```
open (IN, $file) or die;
```

```
($first, $extension) = split /\./, $file;
```

```
$rank = 0;
```

```
foreach (<IN>)
```

```
{
```

```
    chomp $_;
```

```
    next unless $_ =~ m/^t*(\d+)\t*/;
```

```
    $y[$rank] = $1;
```

```
    $rank++;
```

```
}
```

```
@y = sort {$b <=> $a} @y;
```

```
foreach(.01,.02,.03,.04,.05,.06,.07,.08,.09,.1,.11,.12,.13,.14,.15,.20,.25,.30,.35,.40,.45,.50)
```

```
{
```

`$amount = int($rank * $ _);#determines the index of the given percentage of the data. IE: inded for the first 2% of the data.`

`#or the number of indexes you need to change that percentage of the data.`

```
for($l=0; $l<$rank; $l++)
```

```
{
```

```
    $new[$l]=$y[$l];#duplicate array.
```

```
}
```

```
for($i = 0; $i<$amount; $i++)#create an array of the indexes randomnly chosen.
```

```
{
```

```
    my $np;
```

```
    do
```

```
    {
```

```
        $np = int(rand($rank));
```

```
        #print "chose $np\n";
```

```
    } until ($new[$np] ne " ");
```

```
    $new[$np]=" ";
```

```
    #print "$np success!\n";
```

```
}
```

```
chdir("$first");
```

```
chdir('complete');
```

```
open( OUT, ">$_. "$first".complete!.txt');
```

```
foreach (@new)
```

```
{
```

```
    print OUT $_, "\n";
```

```
}
```

```
close(OUT);
```

```
}
```

```
exit(0);
```

#Chris Nuhn & Doc Sessions & Eric Foster

#6:00 PM 02/16/2010

#research algorithm using Zipf's Powerlaw v4.1

#last edited 8:29pm 04/14/2011

#research.pl

#####

ARGS

#####

1. String File

2. Boolean Create .xls

3. Boolean Print

#####

if (\$#ARGV >= 4) {\$pt2 = \$ARGV[4];} #boolean to determine whether or not you wish to run research.pl on pt2.

if (\$#ARGV >= 3) {\$pt1 = \$ARGV[3];} #here is where you pass the directory IE: words folder, to find dirty part1 data for research.pl to run on.

if (\$#ARGV >= 2) {\$print = \$ARGV[2];}

if (\$#ARGV >= 1) {\$xls = \$ARGV[1];}

if (\$#ARGV >= 0) {\$file = \$ARGV[0]; \$error = "file not found\n";}

if (\$#ARGV < 0) {\$error = "file not given\n";}

if(\$file =~ m/^(.+)\.txt\$/) #used to grab the front of the file name and verify it is a text file.

{

 \$first=\$1;

}

if(\$pt1) #if you were given a directory for p1, switch to the appropriate directory.

{

 chdir('data') or die "Can't open \data \$!\n";

 chdir("\$pt1") or die "Can't open \Spt1 \$!\n";

```
}
```

```
if($pt2) #if you were given true for pt2 data, then switch to that directory and run.
```

```
{
```

```
    chdir('complete') or die "Can't open \dir\pt $!\n";
```

```
}
```

```
open( OUT, ">$first"."research.txt");
```

```
foreach("",.01,.02,.03,.04,.05,.06,.07,.08,.09,.1,.11,.12,.13,.14,.15,.20,.25,.30,.35,.40,.45,.50)
```

```
{
```

```
    open (IN, $_.file);#removed his error since it was always thrown and was just ugly.
```

```
    $rank = 0;
```

```
    foreach (<IN>)
```

```
    {
```

```
        chomp $_;
```

```
        next unless $_ =~ m/^t*(\d+)$/;
```

```
        $y[$rank] = $_;
```

```
        $rank++;
```

```
    }
```

```
@y = sort {$b <=> $a} @y;
```

```
$rank = 0;
```

```
foreach (@y)
```

```
{
```

```
    $logy[$rank] = log10($_);
```

```
    $rank++;
```

```
}
```

```

foreach (1..($#y+1))
{
    $x[$_ - 1] = $_;
    $logx[$_ - 1] = log10($_);
}
print OUT (correlation(\@logx, \@logy, $print) . "\t" . slope(\@logx, \@logy, $print) . "\n");
}

```

```
#####
```

```
#          SUBROUTINES          #
```

```
#####
```

```
sub mean
```

```

{
    my ($ref_) = @_ ;
    my $size = @{$ref_};
    my $mean = 0;
    foreach (@{$ref_})
    {
        $mean += $_;
    }
    return ($mean / $size);
}

```

```
sub toxls()
```

```

{
    ($first, $extension) = split /\./, $file;
    open( OUT, ">$first"."research.txt");
    foreach (@re)
    {

```

```

        print OUT "$_ \n";
    }
}

sub sd
{
    my ($ref_) = @_;
    my $size = @{$ref_};
    my $mean = mean($ref_);
    my $sqtotal = 0;
    foreach (@{$ref_})
    {
        $sqtotal += sqr($_ - $mean);
    }
    return sqrt($sqtotal / $size);
}

sub correlation
{
    my ($ref_x, $ref_y, $print) = @_;
    my $n = @{$ref_x};
    my $sdx = sd($ref_x);
    my $sdy = sd($ref_y);
    my $xbar = mean($ref_x);
    my $ybar = mean($ref_y);
    my $sum = 0;
    foreach (0..($n - 1))
    {
        $sum += ((@{$ref_x}[$_] - $xbar)*(@{$ref_y}[$_] - $ybar));
    }
}

```

```

    my $corr = ($sum / (($n) * $sdx * $sdy));
    if($print) {print "Correlation:\nr: $corr\n";}
    return $corr;
}

```

```
sub sqr
```

```

{
    return (@_[0] * @_[0]);
}

```

```
sub log10
```

```

{
    my ($num) = @_;
    if($num != 0){return (log($num) / log(10));}
}

```

```
sub slope
```

```

{
    my ($ref_x, $ref_y, $print) = @_;
    my $n = @{$ref_x};
    my $sx = 0;
    my $sy = 0;
    my $sxy = 0;
    my $sxx = 0;
    my $syy = 0;
    foreach(0..(@{$ref_x} - 1))
    {
        $sx += @{$ref_x}[$_];
        $sy += @{$ref_y}[$_];
        $sxy += (@{$ref_x}[$_] * @{$ref_y}[$_]);
    }
}

```

```

    $sxx += (@{$ref_x}[$_] * @{$ref_x}[$_]);
    $syy += (@{$ref_y}[$_] * @{$ref_y}[$_]);
}

my $m = (($n * $sxy) - ($sx * $sy)) / (($n * $sxx) - sqrt($sx));

if($print) {print "\nSlope: $m"};

return ($m);
}

#Eric Foster & Doctor Sessions

#7:00 PM 04/14/2011

#execution script to run dirty.pl, dirty-pt2.pl, and research.pl over data

#script takes no parameters.

#execute.pl

print "starting the execution...\n";

opendir DIR,'C:\Users\Eric\Desktop\NewProjectAttempt\data\cities\complete' or die "open failed : $!\n";
#just setting the directory for where data will be.

for(readdir DIR)
{
    if($_ =~ m/\.txt$/)# if the file is indeed a txt file.
    {
        `perl C:/Users/Eric/Desktop/NewProjectAttempt/research.pl $_ 0 0 cities 1`;
        {
            `perl C:/Users/Eric/Desktop/NewProjectAttempt/complete.pl $_`;#execute
complete.pl over all data sets.

            print "execution on $_ finished!";
        }

        `perl C:/Users/Eric/Desktop/NewProjectAttempt/dirty.pl $_`; #execute dirty.pl over all files
in directory.

        `perl C:/Users/Eric/Desktop/NewProjectAttempt/dirty-pt2.pl $_`; #execute dirty-pt2.pl over
all files in directory.

        ($first, $extension) = split /\./, $_;#split the file #split off the file name.

```


Sdir2 = 'C:\Users\Eric\Desktop\NewProjectAttempt\data\'."\$first\'.'complete'; #save the directory that will be named after the first part of the filename.

opendir(DIR2, Sdir2) or die "open failed : \$!\n";

for(readdir DIR2)

{

if(\$_ =~ m/\.txt\$/) #make sure you execute only on text files and not folders.

{

`perl C:/Users/Eric/Desktop/NewProjectAttempt/research.pl \$\_ 0 0 \$first 1` #run research.pl across all data in directory.

}

}

closedir DIR2 or die "close failed: \$!\n";

Sdir3 = 'C:\Users\Eric\Desktop\NewProjectAttempt\data\'."\$first\'.'dirtypt'; #store directory where dirtypt2 data will be stored per data set.

opendir(DIR3, Sdir3) or die "open failed : \$!\n";

for(readdir DIR3)

{

if(\$_ =~ m/\.txt\$/) #make sure it is a text file.

{

`perl C:/Users/Eric/Desktop/NewProjectAttempt/research.pl \$\_ 0 0 \$first 1` #execute across all dirtypt2 data in directory.

}

}

closedir DIR3 or die "close failed: \$!\n";

}

}

closedir DIR or die "close failed : \$!\n";

print "execution finished!\n";

exit(0);

#dirty-pt2.pl

#created by Chris Nuhn

if (\$#ARGV >= 0) {\$file = \$ARGV[0]; \$error = "\$file not found\n";}

if (\$#ARGV < 0) {\$error = "file not given\n";}

chdir("data");#added for ease of use.

open (IN, \$file) or die \$error;

(\$first, \$extension) = split /\./, \$file;

\$rank = 0;

foreach (<IN>)

{

chomp \$_;

next unless \$_ =~ m/^(d+)\t*/;

\$y[\$rank] = \$_;

\$rank++;

}

@y = sort {\$b <=> \$a} @y;

foreach(@y)

{

\$mean += \$_;

}

\$mean = \$mean / \$rank;

\$stopvalue = int(rand((\$y[0] * 1.1)));

```

foreach(.01,.02,.03,.04,.05,.06,.07,.08,.09,.1,.11,.12,.13,.14,.15,.20,.25,.30,.35,.40,.45,.50)
{
    $amount = int($rank * $_);

    for($i = 0; $i < $amount; $i++)
    {
        $new[$i] = rand($stopvalue);
    }

    foreach (@new)
    {
        $y[rand($rank)] = int($_);
    }

    chdir("$first");#added for ease of use.
    chdir("dirtypt");#added for ease of use.
    open( OUT, ">$_"."pt2"."$file");
    foreach (@y)
    {
        print OUT $_, "\n";
    }
}

```

#dirty.pl

#Created by Chris Nuhn

```
if ($#ARGV >= 0) {$file = $ARGV[0]; $error = "$file not found\n";}
```

```
if ($#ARGV < 0) {$error = "file not given\n";}
```

```
chdir("data");#added for ease of use.
```

```

open (IN, $file) or die;

($first, $extension) = split /\./, $file;

$rank = 0;

foreach (<IN>)
{
    chomp $_;
    next unless $_ =~ m/^t*(\d+)\t*/;
    $y[$rank] = $1;
    $rank++;
}

@y = sort {$b <=> $a} @y;

foreach(.01,.02,.03,.04,.05,.06,.07,.08,.09,.1,.11,.12,.13,.14,.15,.20,.25,.30,.35,.40,.45,.50)
{
    $amount = int($rank * $_);
    $stopvalue = int($y[0] * 1.1);

    for($i = 0; $i < $amount; $i++)
    {
        $new[$i] = rand($stopvalue);
    }

    foreach (@new)
    {
        $y[rand($rank)] = int($_);
    }

    chdir("$first");#added for ease of use.
    open( OUT, ">$_.$file");

```

```

foreach (@y)
{
    print OUT $_, "\n";
}
}

```

#Chris Nuhn & Doctor Sessions

#6:00 PM 02/16/2010

#Zipf's Powerlaw v3.2

#powerlaw.pl

#####

ARGS

#####

1. String File

2. Boolean Create .xls

3. Boolean Print

#####

if (\$#ARGV >= 2) {\$print = \$ARGV[2];}

if (\$#ARGV >= 1) {\$xls = \$ARGV[1];}

if (\$#ARGV >= 0) {\$file = \$ARGV[0]; \$error = "\$file not found\n";}

if (\$#ARGV < 0) {\$error = "file not given\n";}

#chdir("data") or die "can't find data directory.\n";

open (IN, \$file) or die \$error;

\$rank = 0;

foreach (<IN>)

{

 chomp \$_;

```

    next unless $_ =~ m/^t*(\d+)\t*/;
    $y[$rank] = $1;
    $rank++;
}

```

```
@y = sort {$b <=> $a} @y;
```

```

$rank = 0;
foreach (@y)
{
    $logy[$rank] = log10($_);
    $rank++;
}

```

```

foreach (1..($#y+1))
{
    $x[$_ - 1] = $_;
    $logx[$_ - 1] = log10($_);
}

```

```

correlation(\@logx, \@logy, $print);
if($xls)
{
    toxls();
}

```

```
#####
```

```
#          SUBROUTINES          #
```

```
#####
```

```
sub mean
```

```

{
    my ($ref_) = @_ ;
    my $size = @{$ref_};
    my $mean = 0;
    foreach (@{$ref_})
    {
        $mean += $_;
    }
    return ($mean / $size);
}

sub toxls()
{
    ($first, $extension) = split /\./, $file;
    chdir("\XLS");
    open( OUT, ">$first"."log.xls");
    foreach (0..$#y)
    {
        print OUT $x[$_], "\t", $y[$_], "\t", $logx[$_], "\t", $logy[$_], "\n";
    }
    print "$first"."log.xls created\n";
}

sub sd
{
    my ($ref_) = @_ ;
    my $size = @{$ref_};
    my $mean = mean($ref_);
    my $sqtotal = 0;
    foreach (@{$ref_})

```

```

    {
        $sqtotal += sqrt($_ - $mean);
    }
    return sqrt($sqtotal / $size);
}

```

sub correlation

```

{
    my ($ref_x, $ref_y, $print) = @_;
    my $n = @{$ref_x};
    my $sdx = sd($ref_x);
    my $sdy = sd($ref_y);
    my $xbar = mean($ref_x);
    my $ybar = mean($ref_y);
    my $sum = 0;
    foreach (0..($n - 1))
    {
        $sum += ((@{$ref_x}[$_] - $xbar)*(@{$ref_y}[$_] - $ybar));
    }
    my $corr = ($sum / (($n) * $sdx * $sdy));
    if($print) {print "Correlation:\nr: $corr\n";}
    return $corr;
}

```

sub sqr

```

{
    return (@_[0] * @_[0]);
}

```

sub log10

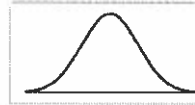

```
{  
    my ($num) = @_;  
    if($num!=0)  
    {  
        return (log($num) / log(10));  
    }  
}
```

Zipf's Law: Completeness and Accuracy

Eric Foster
Advisor: Dr. Valerie Sessions,
April 20, 2011

Background

- We usually think of most data sets as normal distributions.
- Many real-world data actually fits more closely to power law sets.
- Error can be calculated in a normal distribution using Grubb's test

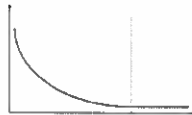


$$g = \frac{\bar{x} - x_{min}}{s}$$

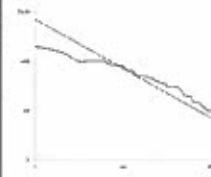
$$g = \frac{x_{max} - \bar{x}}{s}$$

Power Law Distribution

- While a normal distribution lends itself to Grubb's test, a power law distribution doesn't have a means to do this.



Zipfian Distribution



- In Zipfian distributions data is reordered. The data is listed in descending order and the log of the ranks is graphed against the log of the specific values. If the resulting graph is linear, then it fits Zipf's law.

How will we test this data?

- Chris Nuhn's project corrupted the data.
- Only did five data sets.
- My project removed certain percentages of data.
- My project used 12 data sets.
- My project expanded the corrupt data significantly.
- My project also revised code within Chris's script.

Data Used

- Blackouts - the number of customers affected by electrical blackouts in the United States between 1984 and 2002.
- Cities - The population of US cities in the 2000 national Census.
- Crime - Crime activity in a Certain US city.
- Fires - The size in acres of wildfires that occurred in the US on federal land between 1986 and 1996.
- Flares - The peak gamma-ray intensity of solar flares between 1980 and 1989.
- Citations - the number of citations received between publication and June 1997 by scientific papers published in 1981 and listed in the Science Citation Index.
- MOM - the number of species per genus of recent mammals.
- Names - The frequencies of US family names in the 1990 United States Census.
- Quakes - Intensity of earthquakes in California from 1910 to 1992, measured as the maximum amplitude during the quake.
- Terror - the severity of terror attacks globally from February 1968 to June 2006.
- Web - The number of links to websites found in a 1997 web crawl of around 200 million web pages.
- Words - the frequency of unique words in the novel *Moby Dick*.

Data Source:

<http://tuvalu.santafe.edu/~aaronc/powerlaws/data.htm>

- › **Using existing code**
 - › -learning Chris's code
 - › -Very little commenting
 - › -Original scripts had **issues** with data that was zero. Revised it
- › **Format**
 - › -deal with decimal data
 - › -Deal with tabs and spaces misplaced in data
- › **Running Across Data Sets**
 - › -run scripts across multiple files in directory
- › **Large Data sets and Memory Issues**
 - › -couldn't index large data sets because of overflow.
- › **Locating Data**
 - › -data came in illegible formats, some referred to other data unavailable.

[illegible]

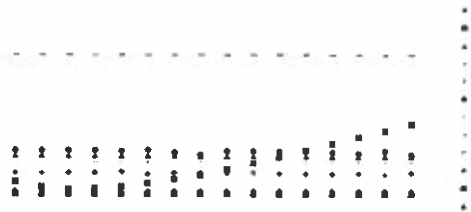
What are
\$rank?
\$amount?
\$new?
\$np?

```
#!/bin/c /Users/eric/Desktop/HowToJoinAccounts/JoinAccounts.c
#include <stdio.h>
#define MAX 100

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <username>\n", argv[0]);
        return 1;
    }
    char username[MAX];
    strcpy(username, argv[1]);
    printf("Username: %s\n", username);
    return 0;
}
```

Unlink simply deletes the old data file.

Results of Dirty Data with a tighter interval



What the data means

-Data with weak p values, meaning they weren't strong power laws and even weaker Zipfian distributions, didn't fit into the data set as well, as was the case with quakesconvert.txt

-When data corruption occurred with both dirty data and removal of data, at about 50% the correlation values jumped significantly.

-completeness doesn't appear to have proved much in the way of identifying outliers. This data will require further analysis to provide insight. Dr. Sessions will continue to examine the results and try to determine a line that can be used to test for outliers.